TECHNICAL REPORT

Intelligent Sudoku Solver Using Constraint Satisfaction Problem (CSP)

Name: Nafis Iqbal Neloy
ID: 0112330617

1. Introduction

Sudoku is a popular constraint-based puzzle that requires assigning values to cells while satisfying a fixed set of constraints. In this project, an intelligent Sudoku solver was developed by modeling Sudoku as a Constraint Satisfaction Problem (CSP).

Instead of using a simple brute-force approach, this implementation uses Artificial Intelligence techniques including constraint propagation, Arc Consistency-3 (AC-3), recursive Backtracking Search, Forward Checking, and the Minimum Remaining Values (MRV) heuristic to efficiently reduce the search space and find valid solutions.

The main objective of this project was to develop an AI-based Sudoku solver capable of solving puzzles with different levels of difficulty by intelligently selecting variables and reducing unnecessary searches.

2. Problem Formulation as CSP

A Sudoku puzzle can be represented as a Constraint Satisfaction Problem (CSP) consisting of three main components:

- Variables
- Domains
- Constraints

2.1 Variables

The Sudoku board contains 81 independent cells. Each cell is considered as a variable.

X= {X1,1,X1,2,.....,X9,9} Each variable represents a specific position in the 9×9 Sudoku grid.

2.2 Domains

Each variable has a possible value domain:

D= {1,2,3,4,5,6,7,8,9}

For pre-filled cells, the domain contains only the given value.

For empty cells, the domain initially contains all possible values from 1 to 9.

The input puzzle is provided as an 81-character string where digits 1-9 represent assigned values and 0 represents unassigned variables.

2.3 Constraints

The Sudoku solution must satisfy three main constraints:

Row Constraint

No number can appear more than once in the same row.

Column Constraint

No number can appear more than once in the same column.

3×3 Sub-grid Constraint

No number can appear more than once inside any 3×3 block.

A valid Sudoku solution requires assigning values to all 81 variables while satisfying all constraints.

3. Algorithm Implementation

3.1 AC-3 Algorithm

Arc Consistency-3 (AC-3) algorithm was implemented as a preprocessing step before applying backtracking search.

The purpose of AC-3 is to reduce the possible values of each variable by removing values that cannot satisfy neighboring constraints.

The AC-3 algorithm works as follows:

1. Initialize a queue containing all arcs between connected variables.
2. Remove an arc from the queue.
3. Apply the REVISE operation.
4. Remove unsupported values from variable domains.
5. If a domain changes, update related arcs.
6. Continue until the queue becomes empty.

AC-3 reduces the search space and improves the efficiency of the solver.

3.2 Backtracking Search

After AC-3 preprocessing, recursive Backtracking Search is used to assign values to remaining variables.

The algorithm follows these steps:

1. Select an unassigned variable.
2. Try possible values from its domain.
3. Check whether the assignment satisfies all constraints.
4. Continue recursively.
5. If failure occurs, undo the assignment and try another value.

This approach avoids unnecessary exploration of invalid states.

3.3 Minimum Remaining Values (MRV) Heuristic

The Minimum Remaining Values (MRV) heuristic improves the variable selection strategy.

Instead of selecting variables sequentially, MRV chooses the variable with the smallest remaining domain.

Advantages of MRV:

- Selects the most constrained variable first.
- Reduces unnecessary searching.
- Decreases recursive calls.
- Improves solving efficiency.

4. Object-Oriented Design

The solver was implemented using an object-oriented architecture.

The main class used in the implementation is:

SudokuCSP Class

Responsibilities of this class include:

- Creating Sudoku variables.
- Maintaining variable domains.
- Generating neighboring variables.
- Checking constraints.
- Performing AC-3 preprocessing.
- Applying forward checking.
- Executing backtracking search.

This structure provides a clear representation of the CSP model and improves code organization.

5. Input and Output Handling

The program reads Sudoku puzzles from:

Input File: puzzles.txt

Each puzzle is represented as an 81-character string.

After solving, the program generates:

Output File: solutions.txt

The output file contains completely solved Sudoku boards represented as 81-character strings without any zero values.

6. Experimental Results

The solver was tested using multiple Sudoku puzzles.

The program identified the hardest puzzle based on the highest number of expanded nodes.

Puzzle Analysis

Puzzle	Nodes Expanded
Puzzle 1	82
Puzzle 2	37038
Puzzle 3	11408

The hardest puzzle was selected as:

Puzzle 2

Maximum Nodes Expanded:

37038

7. Ablation Study

To evaluate the effect of the MRV heuristic, the hardest puzzle was executed under two different conditions:

1. Without MRV heuristic
2. With MRV heuristic

Performance Comparison

Method	Execution Time (seconds)	Nodes Expanded	Backtracks
Without MRV	231.5135	17028968	17028886
With MRV	0.8711	37038	36956

8. Analysis of Ablation Study

The experimental results clearly demonstrate the effectiveness of the MRV heuristic.

Without MRV:

- The solver expanded approximately 17 million nodes.
- Execution time was more than 231 seconds.
- A large number of backtracking operations were required.

With MRV:

- The solver expanded only 37038 nodes.
- Execution time decreased to less than one second.
- The number of backtracking operations was significantly reduced.

The result shows that MRV greatly improves CSP solving performance by selecting the most constrained variable first and reducing unnecessary exploration.

9. Conclusion

This project successfully developed an Artificial Intelligence based Sudoku solver using the Constraint Satisfaction Problem approach.

The combination of AC-3 preprocessing, Backtracking Search, Forward Checking, and MRV heuristic allowed the solver to efficiently solve Sudoku puzzles.

The ablation study proved that MRV provides a significant improvement in performance by reducing execution time, node expansion, and backtracking operations.

This project demonstrates the effectiveness of CSP-based Artificial Intelligence techniques for solving complex combinatorial problems.